



Hacking with Hashcat - A Practical Guide

HASHCAT

Hashing is often confused with encryption. A simple difference is that hashed data is not reversible. Encrypted data can be reversed using a key. This is why applications like Telegram use encryption while passwords are hashed.

In this article, we will look at installing and working with Hashcat. Hashcat is a simple but powerful command line utility that helps us to – you guessed it – crack hashes.

We will first start by looking at how hashing works in detail.

Note: All my articles are for educational purposes. If you use this information illegally and get into trouble, I am not responsible. Always get permission from the owner before scanning / brute-forcing / exploiting a system.

What is Password Hashing?

Hashing is the process of converting an **alphanumeric** string into a fixed-size string by using a hash function. A hash function is a mathematical function that takes in an input string and generates another alphanumeric string.

Hashing



How hashing works

There are many hashing algorithms like MD5, SHA1, and so on. To learn more about different hashing algorithms, [you can read the article here](#).

The length of a hash is always a constant, irrespective of the length of the input. For example, if we use the MD5 algorithm and hash two strings like “Password123” and “HelloWorld1234”, the final hash will have a fixed length.

Here is the MD5 hash for “Password123”.

```
42f749ade7f9e195bf475f37a44cafc
```

If we use the input string as “HelloWorld1234”, this will be the result:

```
850eaebd5c4bb931dbb2bbcf7994c021
```

Now there is a similar algorithm called encoding. A popular encoding algorithm is base64. Here is how the same “Password123” will look if we encode it with base64:

```
UGFzc3dvcmQxMjM=
```

So what is the difference between hashing and encoding? When we encode a string, it can be easily decoded to get the source string. But if we hash a string, we can never get to the source string (maybe with quantum computers, but that's another topic for discussion).

Hashing and encoding have different use cases. We can apply encoding to mask/simplify strings while hashing is used to secure sensitive data like passwords.

If hashes are not reversible, how would we compare the strings? Simple – we compare the hashes.

When we signup for a website, they will hash our password before saving it (hopefully!). When we try to log in again, the same hashing algorithm is used to generate a hash for our input. It is then compared with the original hash saved in the database.

This approach is also what gives rise to hashing attacks. A simple way to attack hashes is to have a list of common passwords hashed together. This list is called a Rainbow table. Interesting name for a table of hashes.

Now that we know how hashing works, let's look at what Hashcat is.

What is Hashcat?

Hashcat is a fast password recovery tool that helps break complex password hashes. It is a flexible and feature-rich tool that offers many ways of finding passwords from hashes.

Hashcat is also one of the few tools that can work with the GPU. While CPUs are great for sequential tasks, GPUs have powerful parallel processing capabilities. GPUs are used in Gaming, Artificial intelligence, and can also be used to speed up password cracking.

Here is the difference between a CPU and a GPU if you want to learn more.

Other notable features of Hashcat include:

- Fully open source.
- Support for more than 200 hashing algorithms.
- Support for Windows, Linux, and Mac.
- Support for cracking multiple hashes in parallel.
- Built-in benchmarking system.

Now that we know what Hashcat is, let's go and install it.

How to Install Hashcat

Hashcat comes pre-installed in Kali and Parrot OS. To install it in Ubuntu / Debian-based systems, use the following command:

```
$ apt install hashcat
```

To install it on a Mac, you can use [Homebrew](#). Here is the command:

```
$ brew install hashcat
```

For other operating systems, a full list of installation instructions can be [found here](#).

Once the installation is done, we can check Hashcat's help menu using this command:

```
$ hashcat -h
```

```
Usage: hashcat [options]... hash[hashfile|hccapxfile [dictionary|mask|directory]]...

- [ Options ] -

Options Short / Long      | Type | Description                                     | Example
=====
-m, --hash-type           | Num  | Hash-type, references below (otherwise autodetect) | -m 1000
-a, --attack-mode         | Num  | Attack-mode, see references below               | -a 3
-V, --version             |      | Print version
-h, --help                |      | Print help
--quiet                  |      | Suppress output
--hex-charset             |      | Assume charset is given in hex
--hex-salt                |      | Assume salt is given in hex
--hex-wordlist             |      | Assume words in wordlist are given in hex
--force                  |      | Ignore warnings
--deprecated-check-disable |      | Enable deprecated plugins
--status                 |      | Enable automatic update of the status screen
--status-json             |      | Enable JSON format for status output
--status-timer            | Num  | Sets seconds between status screen updates to X   | --status-timer=1
--stdin-timeout-abort     | Num  | Abort if there is no input from stdin for X seconds | --stdin-timeout-abort=300
--machine-readable        |      | Display the status view in a machine-readable format
--keep-guessing           |      | Keep guessing the hash after it has been cracked
--self-test-disable       |      | Disable self-test functionality on startup
--loopback               |      | Add new plains to induct directory
--markov-hcstat2          | File | Specify hcstat2 file to use                       | --markov-hcstat2=my.hcstat2
--markov-disable          |      | Disables markov-chains, emulates classic brute-force
--markov-classic          |      | Enables classic markov-chains, no per-position
--markov-inverse          |      | Enables inverse markov-chains, no per-position
-t, --markov-threshold    | Num  | Threshold X when to stop accepting new markov-chains | -t 50
--runtime                | Num  | Abort session after X seconds of runtime           | --runtime=10
--session                 | Str  | Define specific session name                       | --session=mysession
--restore                 |      | Restore session from --session
--restore-disable         |      | Do not write restore file
```

Hashcat help menu

In addition to Hashcat, we will also need a wordlist. A word list is a list of commonly used terms. This can be a password wordlist, username wordlist, subdomain wordlist, and so on.

A popular password wordlist is **rockyou.txt**. It contains a list of commonly used passwords and is popular among pen testers. You can find the Rockyou wordlist under /usr/share/wordlists in Kali Linux.

How to Work with Hashcat

Now that we know what hashing and Hashcat are, let's start cracking some passwords.

Before cracking a hash, let's create a couple of hashes to work with. We can use a site like Browserling to generate hashes for input strings.

Let's create two hashes: A MD5 hash and a SHA1 hash for the string "Password123". I'm using a weak password to help you understand how easy it is to crack these passwords.

Here are the generated hashes for the input strings.

```
MD5 hash -> 42f749ade7f9e195bf475f37a44cafcbb
```

```
SHA1 hash -> b2e98ad6f6eb8508dd6a14cfa704bad7f05f6fb1
```

We can store these hashes under the names md5.txt and sha1.txt to use them when working with Hashcat.

To crack a password using Hashcat, here is the general syntax.

```
$ hashcat -m value -a value hashfile wordlist
```

Let's dissect the syntax. We have used two flags, -m and -a . The -m flag is used to specify the hash type and the -a flag is to specify the attack mode. You can find the list of hash types and attack modes here.

Let's crack our md5 hash first. We will crack this hash using the Dictionary mode. This is a simple attack where we provide a list of words (RockYou) from which Hashcat will generate and compare hashes.

We can specify the hash mode as “md5” using the value 0. But Hashcat can also identify the hash type automatically for common hash algorithms.

For the attack mode, we will be using the dictionary mode (0) using the flag -a. Here is the full command:

```
$ hashcat -m 0 -a 0 md5.txt rockyou.txt
```

Hashcat will quickly find the value for the hash, in this case, “Password123”:

```
Host memory required for this attack: 4 MB

Dictionary cache built:
* Filename..: rockyou.txt
* Passwords.: 14344391
* Bytes.....: 139921497
* Keyspace..: 14344384
* Runtime...: 1 sec

42f749ade7f9e195bf475f37a44cafcfb:Password123
```

Hashcat MD5 crack

Looks simple, doesn't it? Now let's crack our SHA hash. The hash mode value for SHA1 is 100. Here is the command:

```
$ hashcat -m 100 -a 0 sha1.txt rockyou.txt
```

And here is the output from Hashcat:

```
Host memory required for this attack: 4 MB

Dictionary cache hit:
* Filename..: rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace..: 14344384

b2e98ad6f6eb8508dd6a14cfa704bad7f05f6fb1:Password123
```

Hashcat SHA1 crack

Hashcat supports almost all hashing algorithms with various attack modes. Let's look at a few attack modes and see how they work.

Dictionary attack (-a 0)

As we saw in our example above, a dictionary attack is performed by using a wordlist. A dictionary attack is also the default option in Hashcat. The better the wordlist is, the greater the chances of cracking the password.

Combinator attack (-a 1)

The combinator attack will try different combinations of words from our wordlist. For example, if our wordlist contains the words "pass", "123", and "hello", Hashcat will generate the following wordlist.

passpass

pass123

passhello

123pass

123123

123hello

hellopass

hello123

hellohello

As you can see, using a simple wordlist can give us a number of combinations. This attack is great if we know some terms that might be used in the password. Keep in mind that, the larger the initial wordlist, the more complicated the final wordlist gets.

Mask attack (-a 3)

The mask attack is similar to the dictionary attack, but it is more specific. Brute-force approaches like dictionary attacks can take a long time to crack a password. But if we have information regarding the password, we can use that to speed up the time it takes to crack the password.

For example, if we know the length of the password and a few characters that might be in the password, we can generate a custom wordlist with those characters.

The mask attack is out of scope for this article, but you can [learn more about mask attacks here](#).

In addition to these common attack types, there are more attack modes in Hashcat. This includes Hybrid mode, Permutation attack, Rule-based attack, and so on. Each of these modes can be used for specific use cases and to speed up password cracking.

How to Defend Against Hashcat

The first and obvious step is to set strong passwords. The stronger the password is, the harder it is to crack it. You can check if your password has been [exposed to the internet here](#).

A more effective way is to [add salts to password hashes](#). A salt is an additional string added to the existing password so the hash generated is different from the normal hash of a string.

For example, if a string “sdf909” is added to a password “Password123”, Rainbow table attacks will immediately fail since they don't have hashes with the salt added to them.

To crack a salted password, the attacker should know both the hash and salt values. This makes it harder to crack hashes using methods such as Rainbow tables.

We can further strengthen salting by using dynamic salts instead of static salts. We can write a function that generates a salt value for every string making it exponentially harder to crack a salted password.

You can [read this article](#) to learn more about how Salts work in password hashing.

Summary

Hashing is the method of using a mathematical function to generate a random string. It is a one-way function and helps to secure data such as user passwords.

Hashcat is a powerful tool that helps to crack password hashes. Hashcat supports most hashing algorithms and can work with a variety of attack modes.

To enforce security and protect hashes from attacks, use strong passwords and salts before hashing passwords.